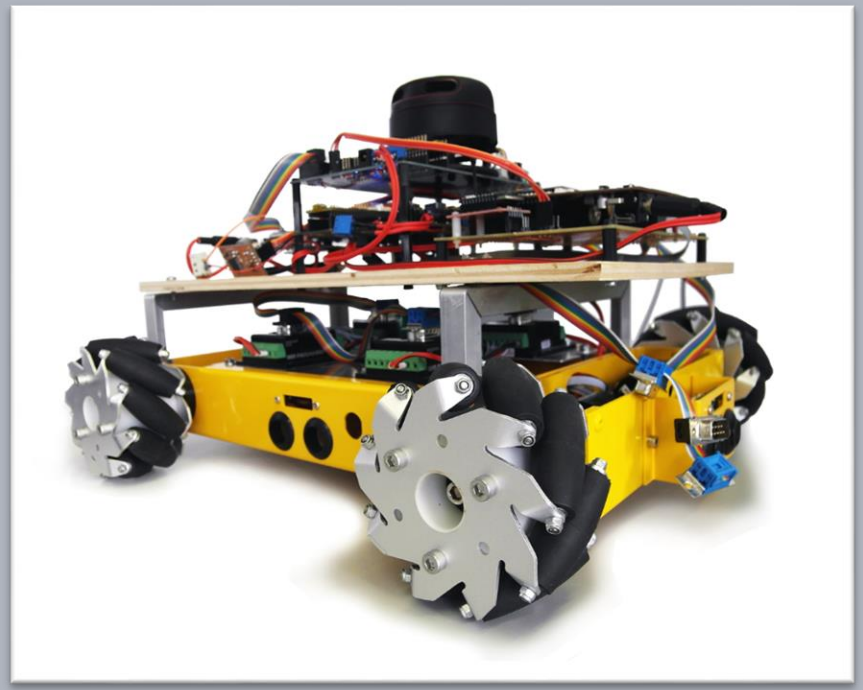
Advanced Embedded Architectures and Applications

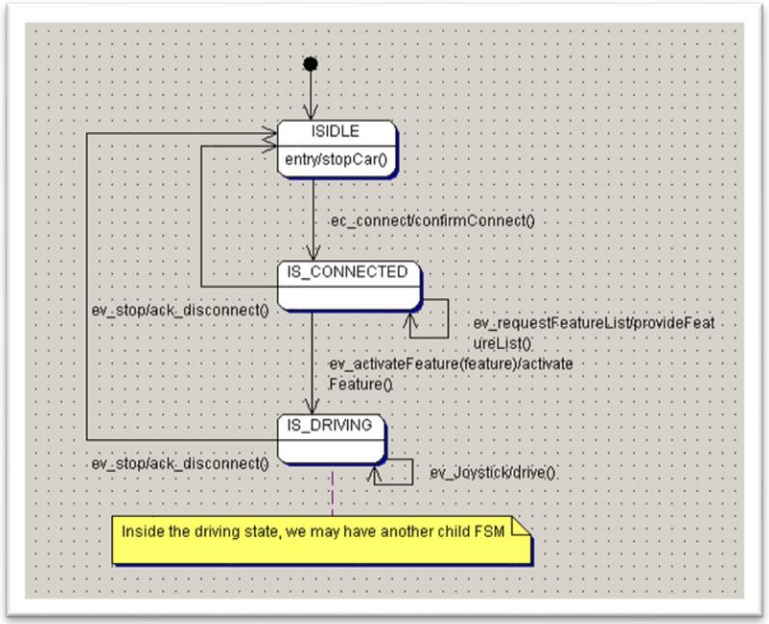
Refactoring and Adding the Connection FSM

Prof. Dr.-Ing. Peter Fromm



Content

- Refactoring the engine
- Stateless versus stateful functions
- Towards Engine Objects
- Improving the decoder accuracy
- Connection State Machine
- State Machine Synchronisation
- State Machine implementation
- Abstract event signals
- Stringnifying of data – the feature list
- Integrating the display
- RTE limitations



Need for refactoring – the engine driver

When designing the initial version of the engine driver, I had the following requirements in mind

- Design an easy to use PID API for a single engine
- Return measured speed and control speed to allow visualization in Matlab
- The ticktime is not measured dynamically, but has to be provided manually.

This resulted in a couple of problems:

- As the measured speed is read inside the PID controller function, we cannot read it as an independent signal. The decoder value is reset upon every read operation to get the delta in the next call.
- Furthermore we have several hacky array datatypes inside the driver to store the values of the engines independently
- The signal concept was not really considered when designing the API

Stateful versus stateless drivers

Many driver function are implemented as stateless functions, i.e. they will always deliver the current value

- ADC_Read()
- PIN_Read()
- PWM_Set()

Some other functions however contain a (hidden) state

- UART_read()

In the case of UART_read() the last byte will be deleted from the internal hardware FIFO. I.e. calling UART_read() twice will deliver different values. The read operation performs an internal state change!

How about our getspeed function?

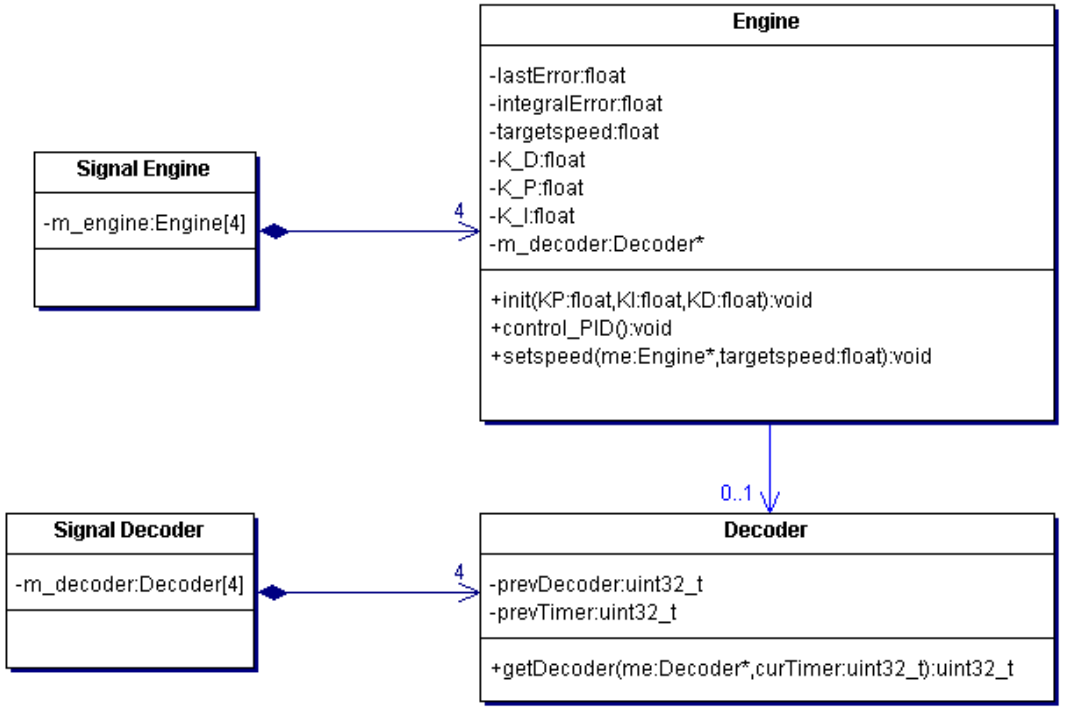
```c
/**
 * Get's the engine speed from the decoder
 * \param uint16_t ticktime : IN - The ticktime in [ms]
 * \param sint16_t speed    : OUT - The engine speed in RPM
 * \return RC_SUCCESS
 */
RC_t ENG_GetSpeed(ENG_id_t id, uint16_t ticktime, sint16_t *const currentSpeed)
{
    sint32_t currentDecoder = 0;
    switch (id)
    {
        case ENG_RL : currentDecoder = RL_QuadDec_GetCounter(); break;
        case ENG_RR : currentDecoder = RR_QuadDec_GetCounter(); break;
        case ENG_FL : currentDecoder = FL_QuadDec_GetCounter(); break;
        case ENG_FR : currentDecoder = FR_QuadDec_GetCounter(); break;
        default: break;
    }



    //Correct sign bit if needed
    currentDecoder *= ENG_getDecoderOrientation(id);

    sint32_t deltaDecoder = currentDecoder - ENGINE_decoder[id];
    ENGINE_decoder[id] = currentDecoder;

    //Debug story
    //DET_Toggle();
    //LOG_I("ENG_GetSpeed"," decoder = %d delta = %d",currentDecoder, deltaDecoder);

    //Ticktime in [ms]. CurrentSpeed in [rpm]
    *currentSpeed = (sint16_t)((deltaDecoder * 60000)/(ticktime * ENG_DECODER_RATIO * ENG_ENGINE_RATIO));

    return RC_SUCCESS;
}
```

State change!

If we read the speed out of sync with the predefined ticktime, the result will be wrong!

Design improvement (Ideas)

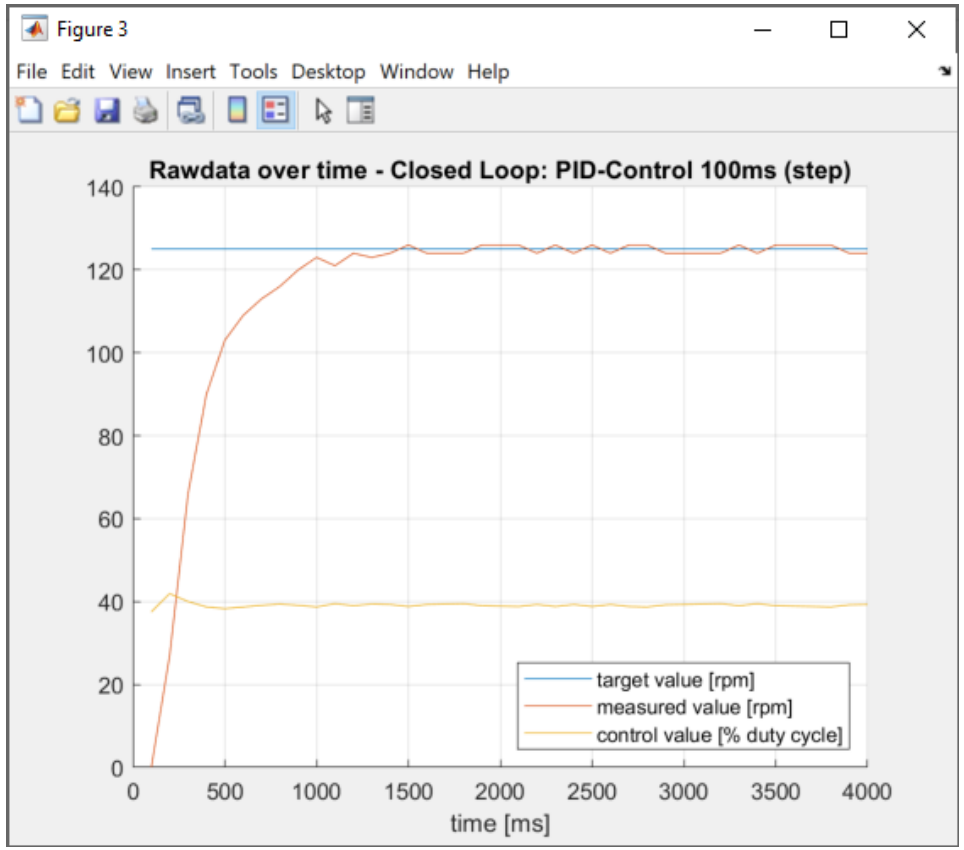
- We create an object oriented design
- We separate updating the decoder value from reading the decoder value
- Instead of providing the ticktime from the outside (very fragile), we provide the absolute time
- Inside the driver, we store the previous absolute time
- Instead of the enum id, we provide a decoder object



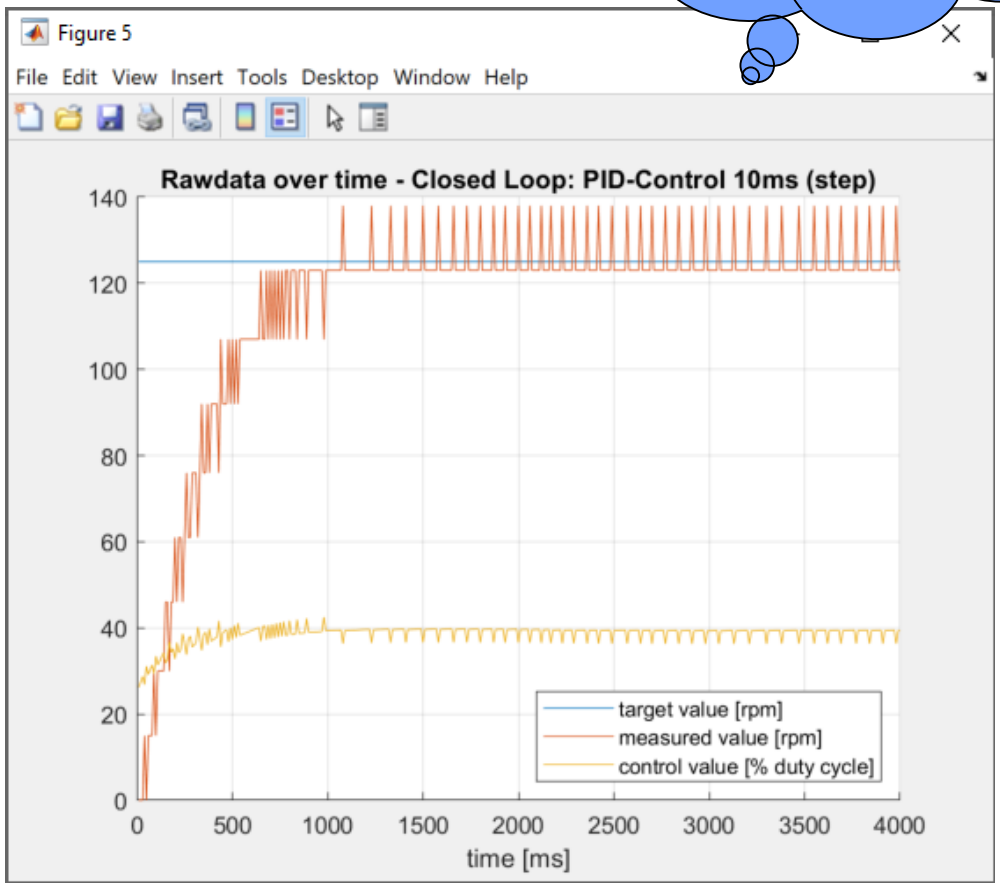
Remember this problem

Precision of the decoder!

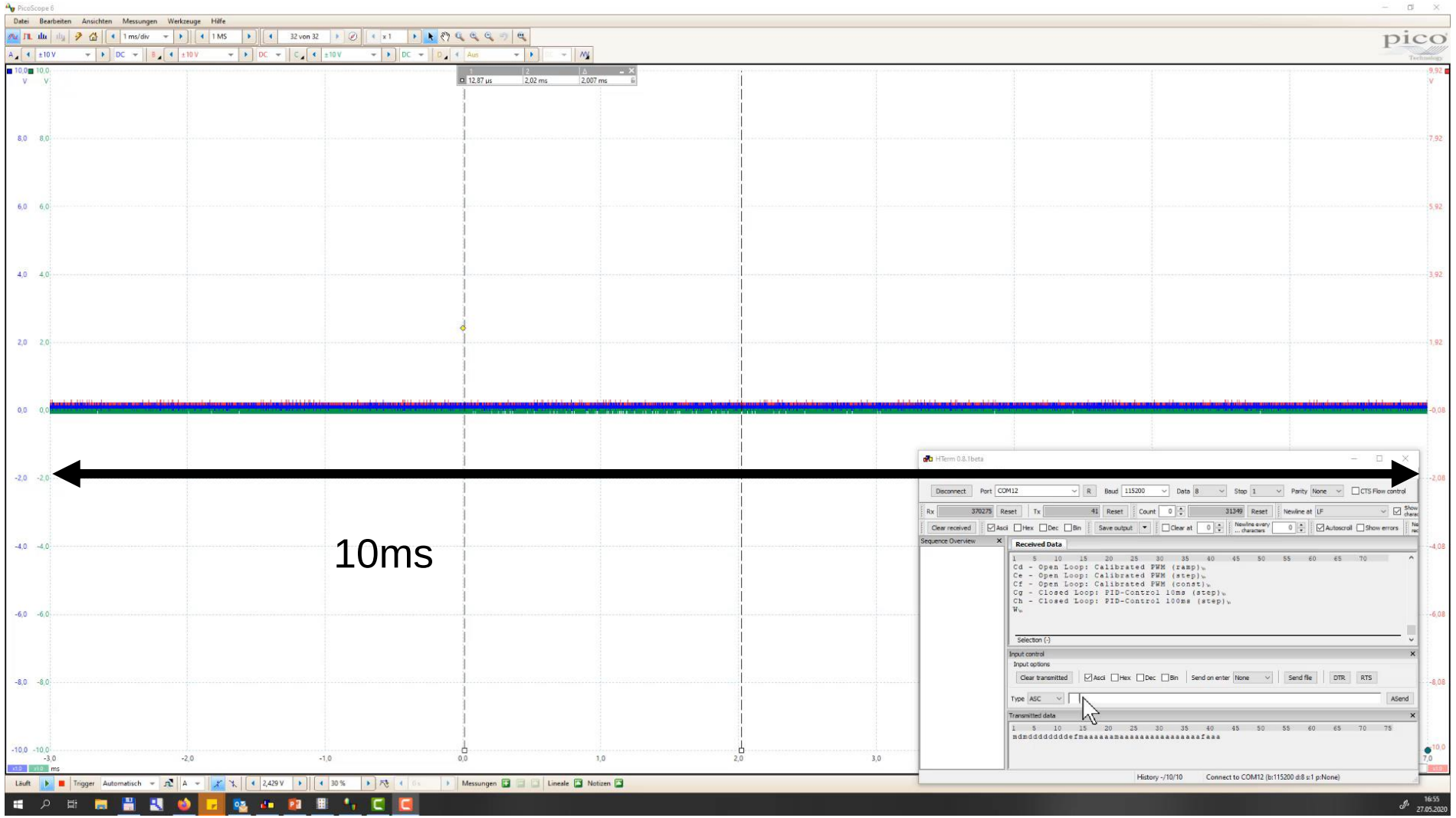
Sample Time 100ms



Sample Time 10ms

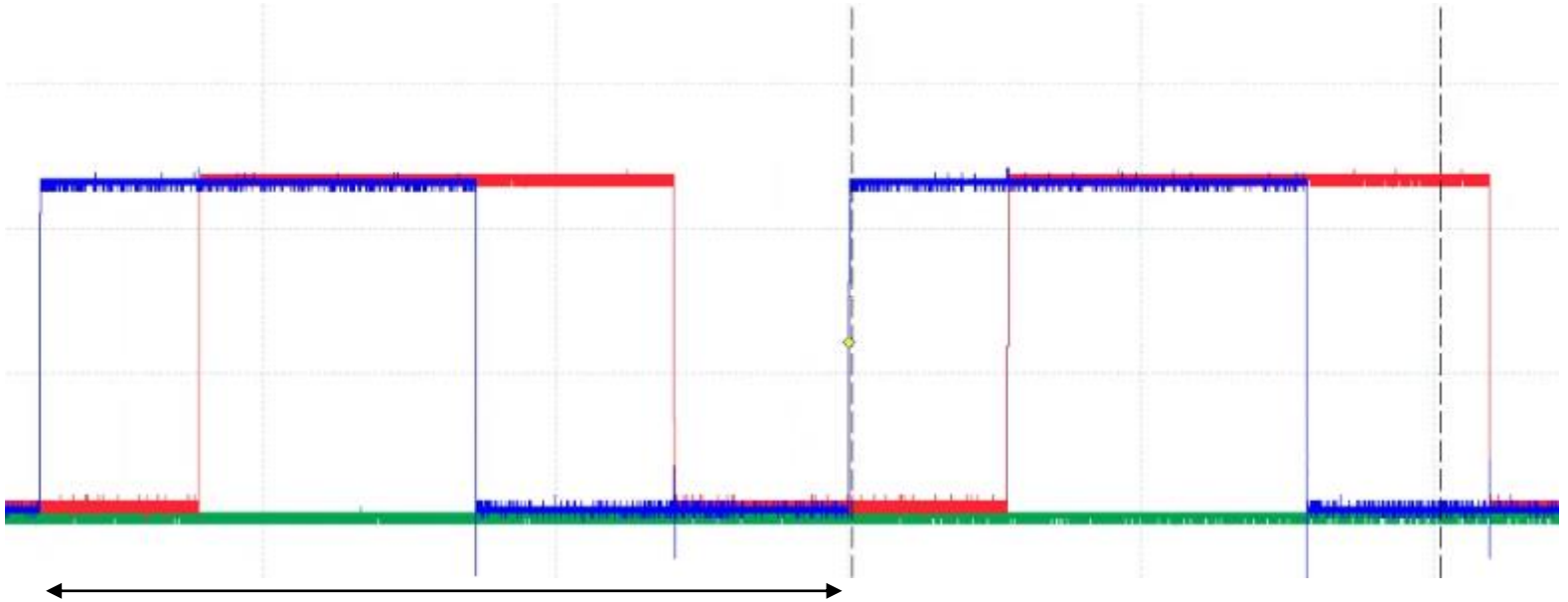


Decoder Accuracy

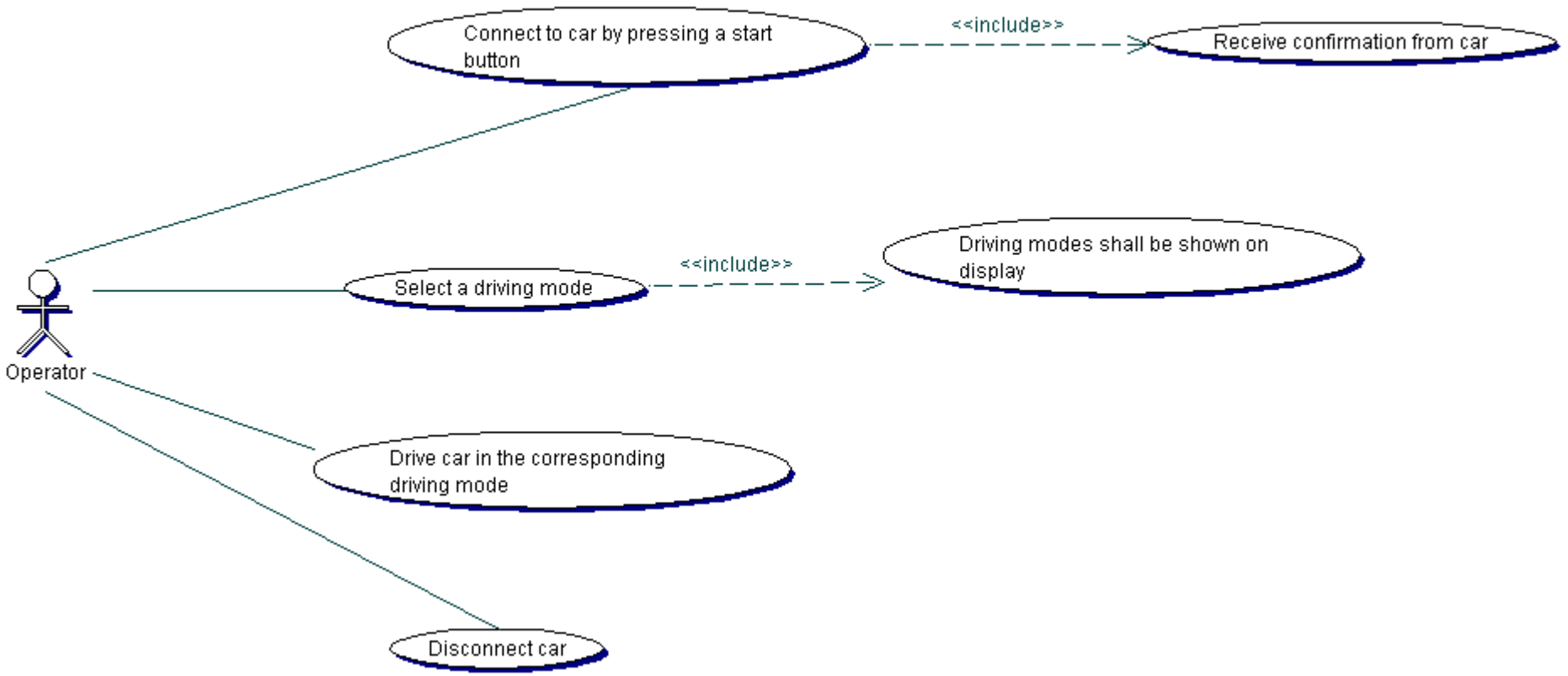


Improving the accuracy for low speeds

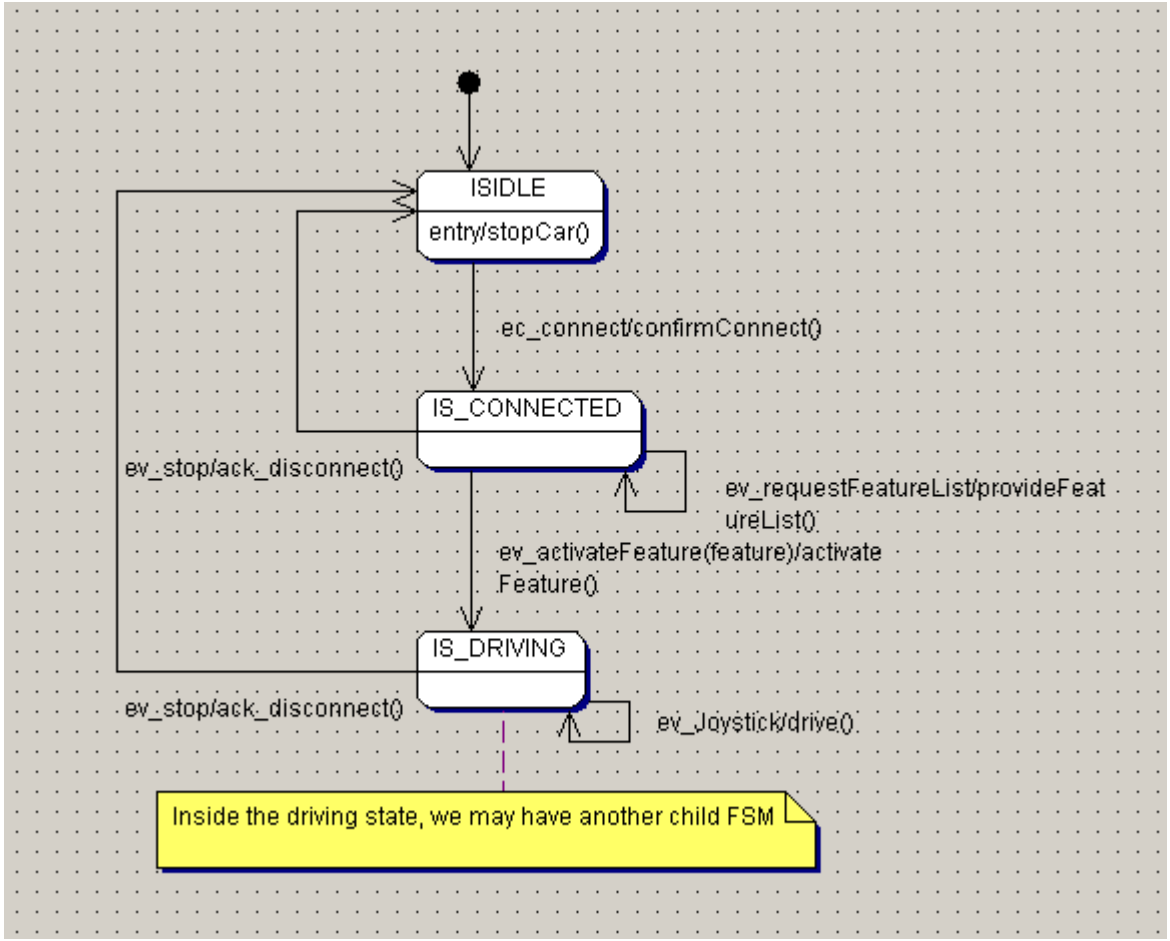
- Instead of counting pulses per timeslot, we might as well measure the time between 2 edges
- Low speed ➔ Time measurement will be better
- High Speed ➔ Edge Detection will be better
- For toggling between the 2 modes, a hysteresis function is recommended



Connection Use Case



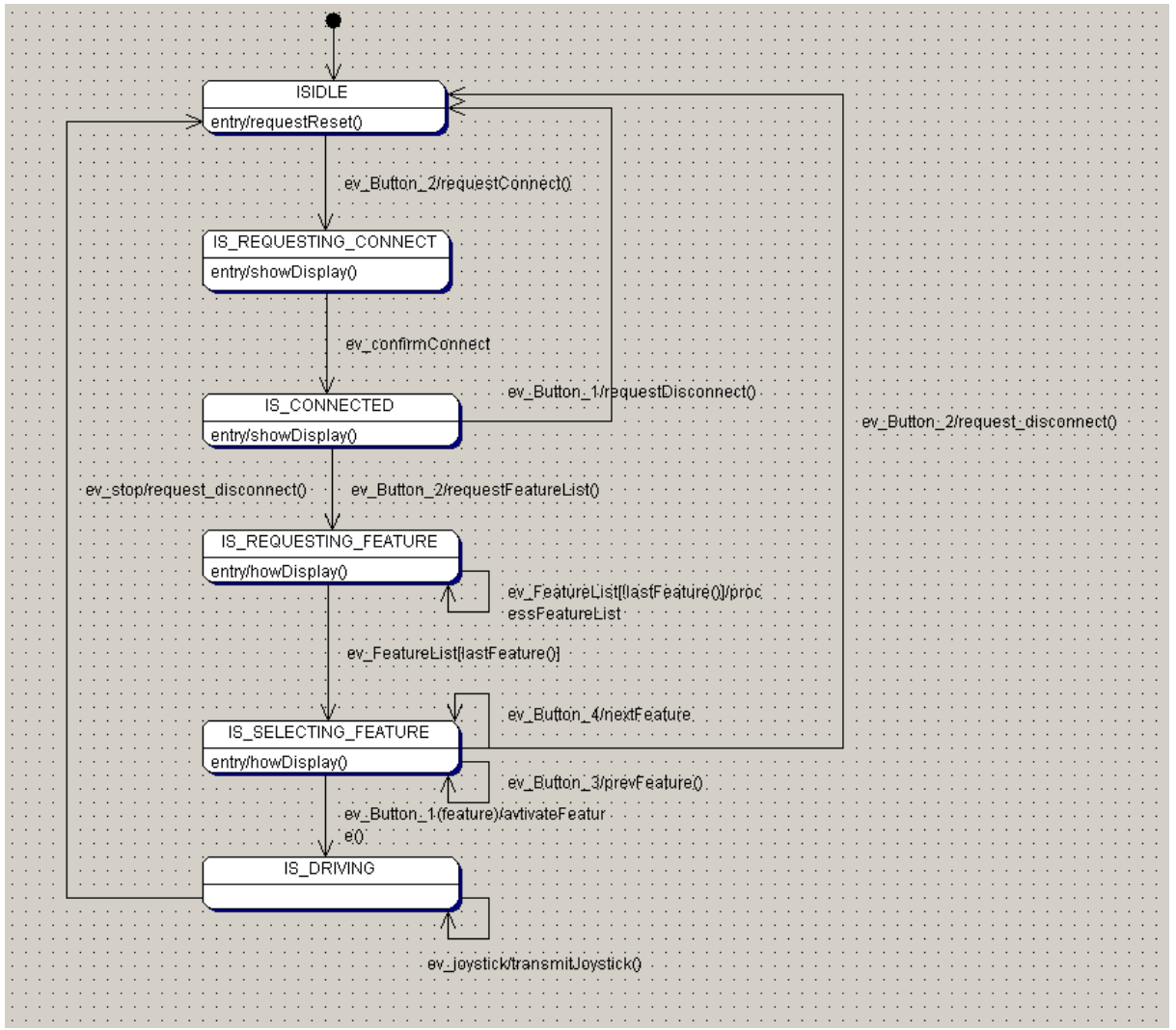
State Machine - Car



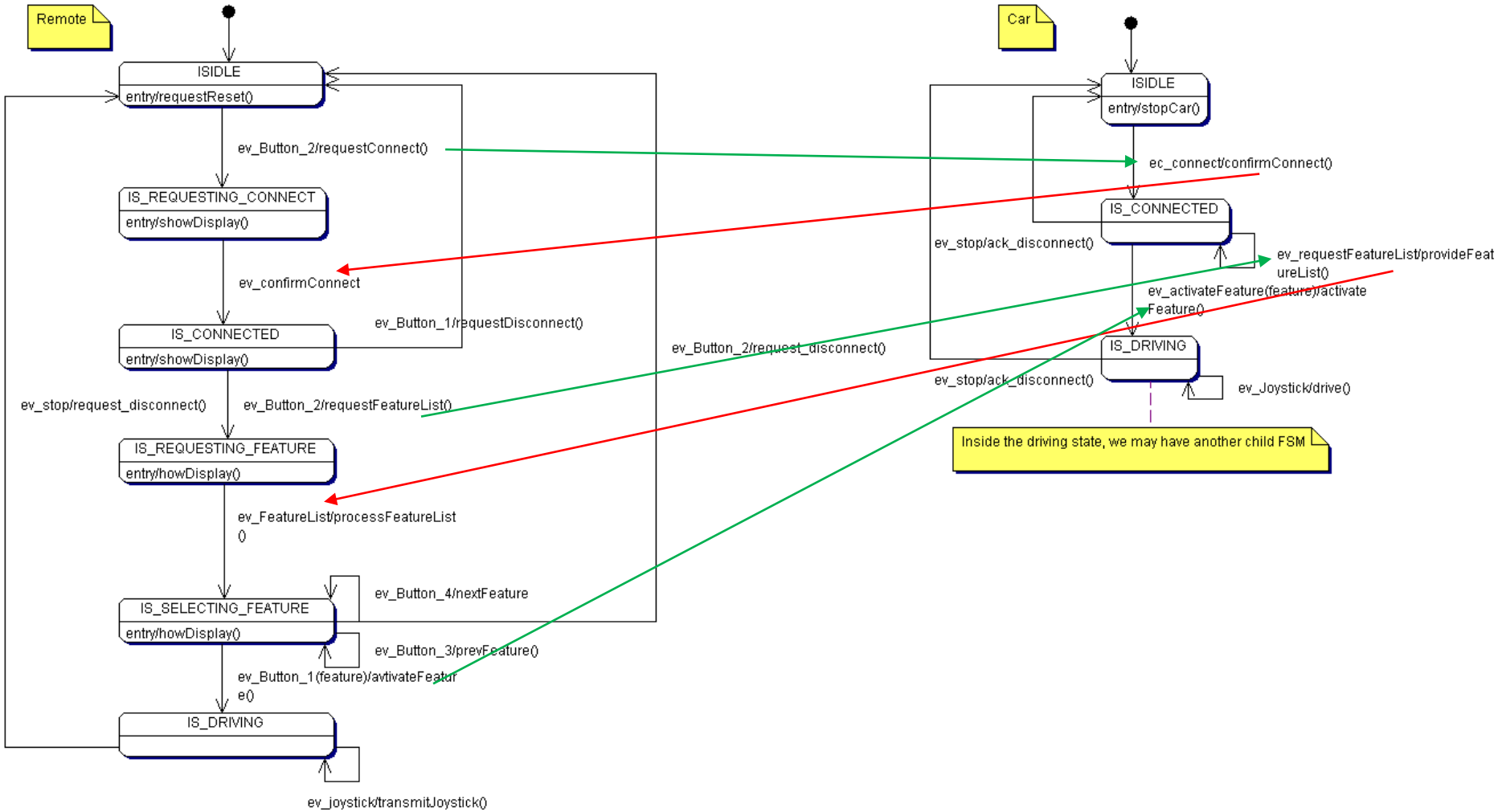
Storyline on the remote side

- The remote control will request a connection when button 2 is pressed
- The connection will be shown on the display
- By pressing button 2 again, the available list of features is requested
- The received feature will be shown on the display
- Using button 3 and 4 the user can navigate through the list of features
- By pressing button 2 again, the selected feature is activated
- Pressing button 1 at any time will issue a disconnect command

Translated into a state machine on Remote side…



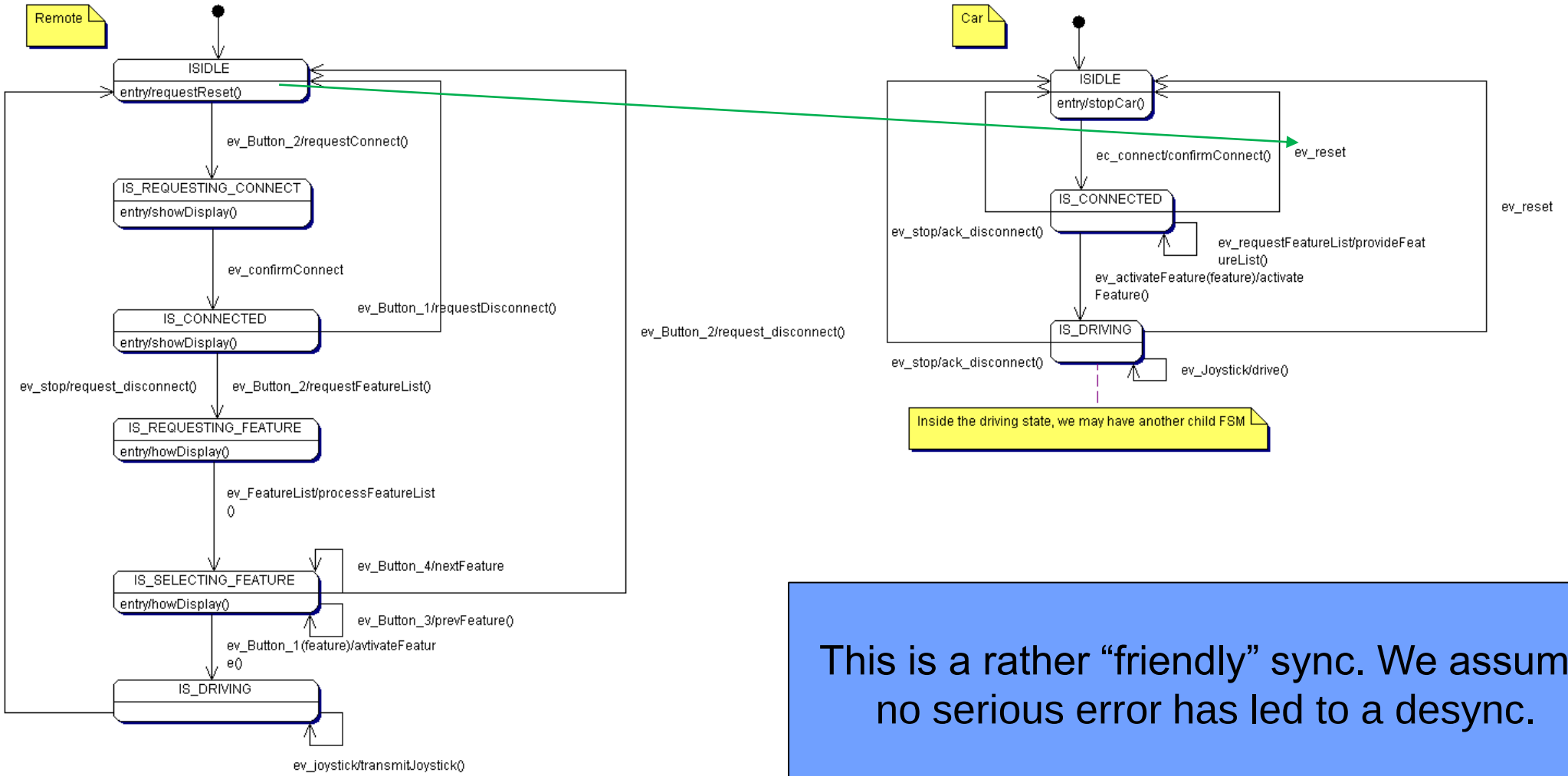
The state machines are not really independent…



Implementation Strategy

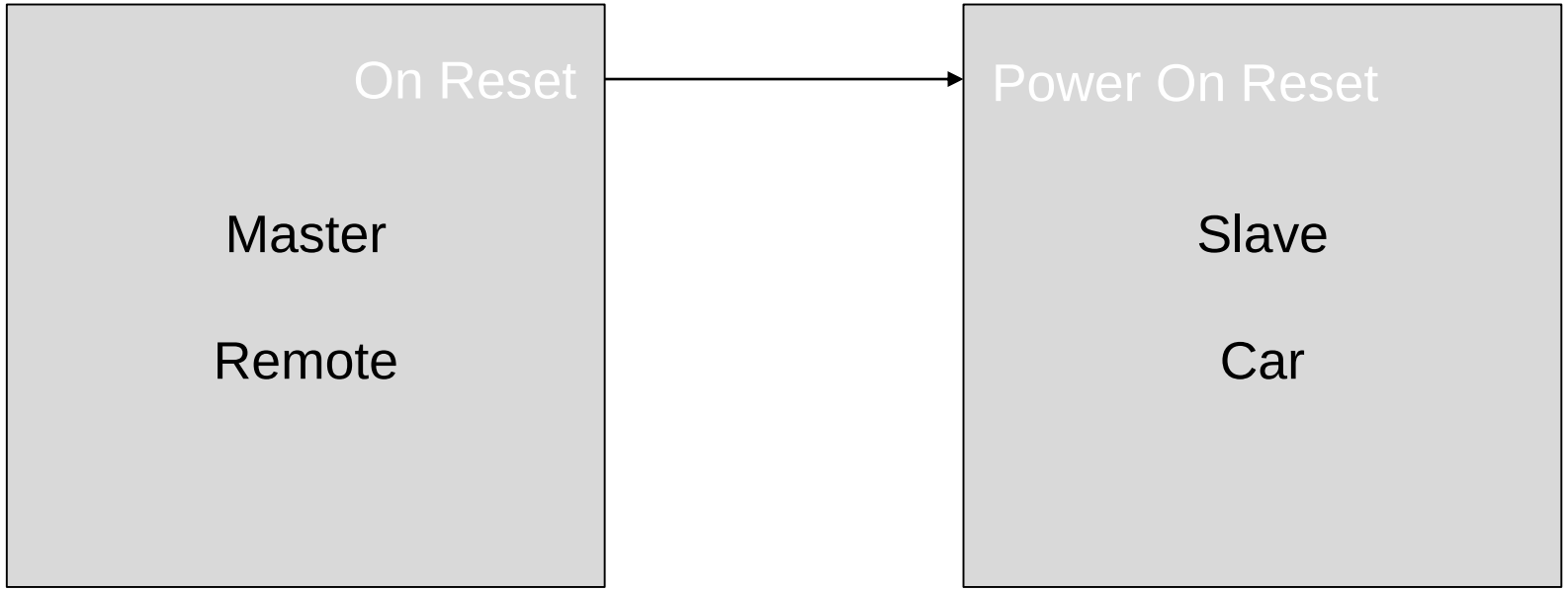
Step	Description	Effort
1	Create state machine design	60min
2	Implement state machine framework and tables, test and debug	60min
3	Create action declarations, events and state enums	30min
4	Add events, guards and actions to state tables	30min
5	Write converters protocol2event and event2protocol	30min
6	Implement first actions and test initial transitions, test and debug	60min
7	Implement feature list parser and packer, test and debug, refactoring	90min
8	Add Joystick protocol, test and debug	30min
9	Integrate display	30min
	Total	7 hours

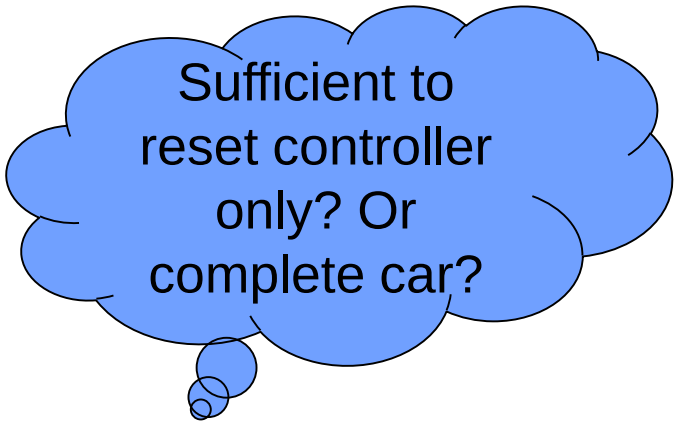
Device synchronization is required



This is a rather “friendly” sync. We assume no serious error has led to a desync.

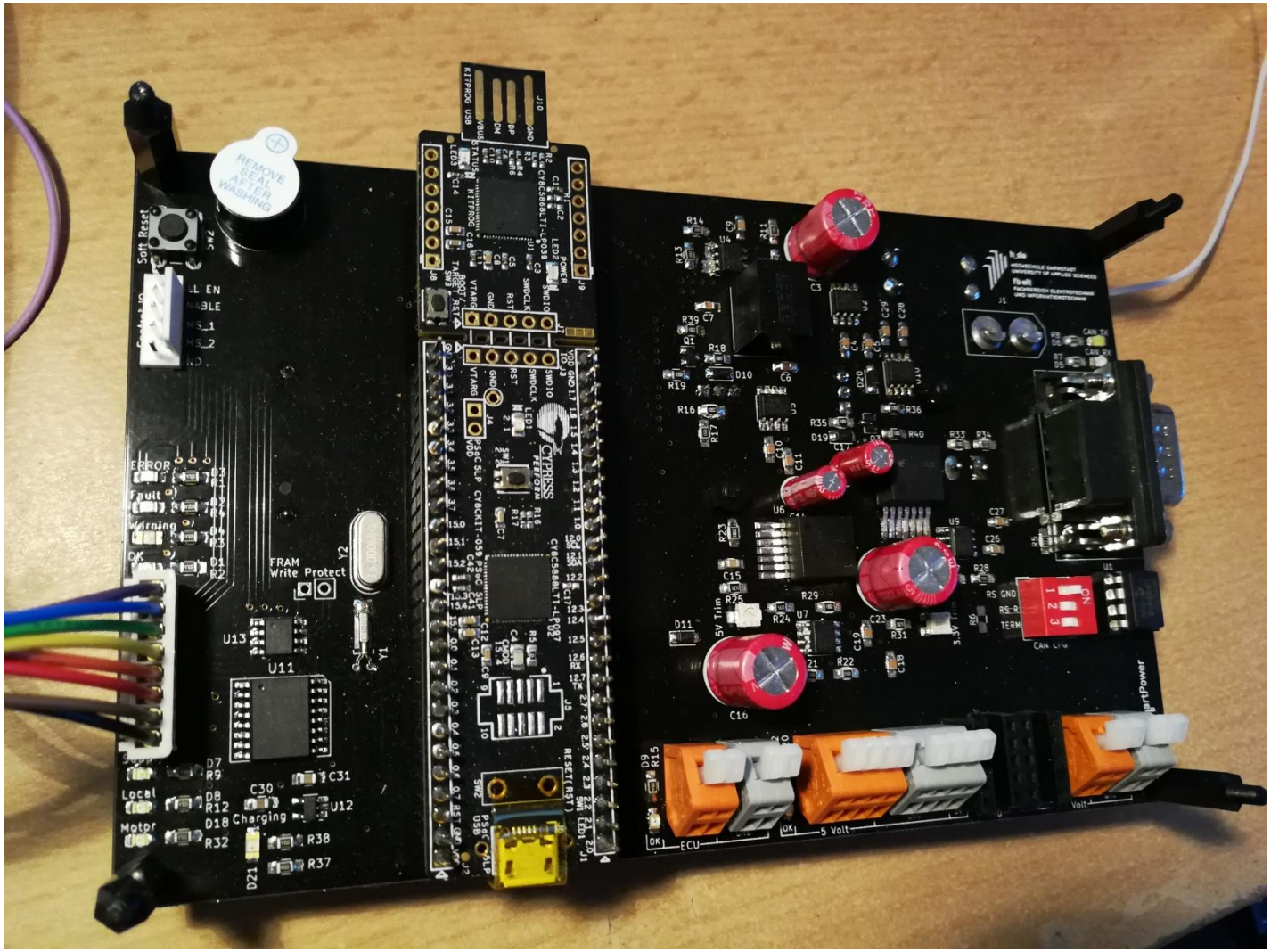
Hard sync





Should be uni-directional from master to slave to avoid reset loops!

SmartPower Board



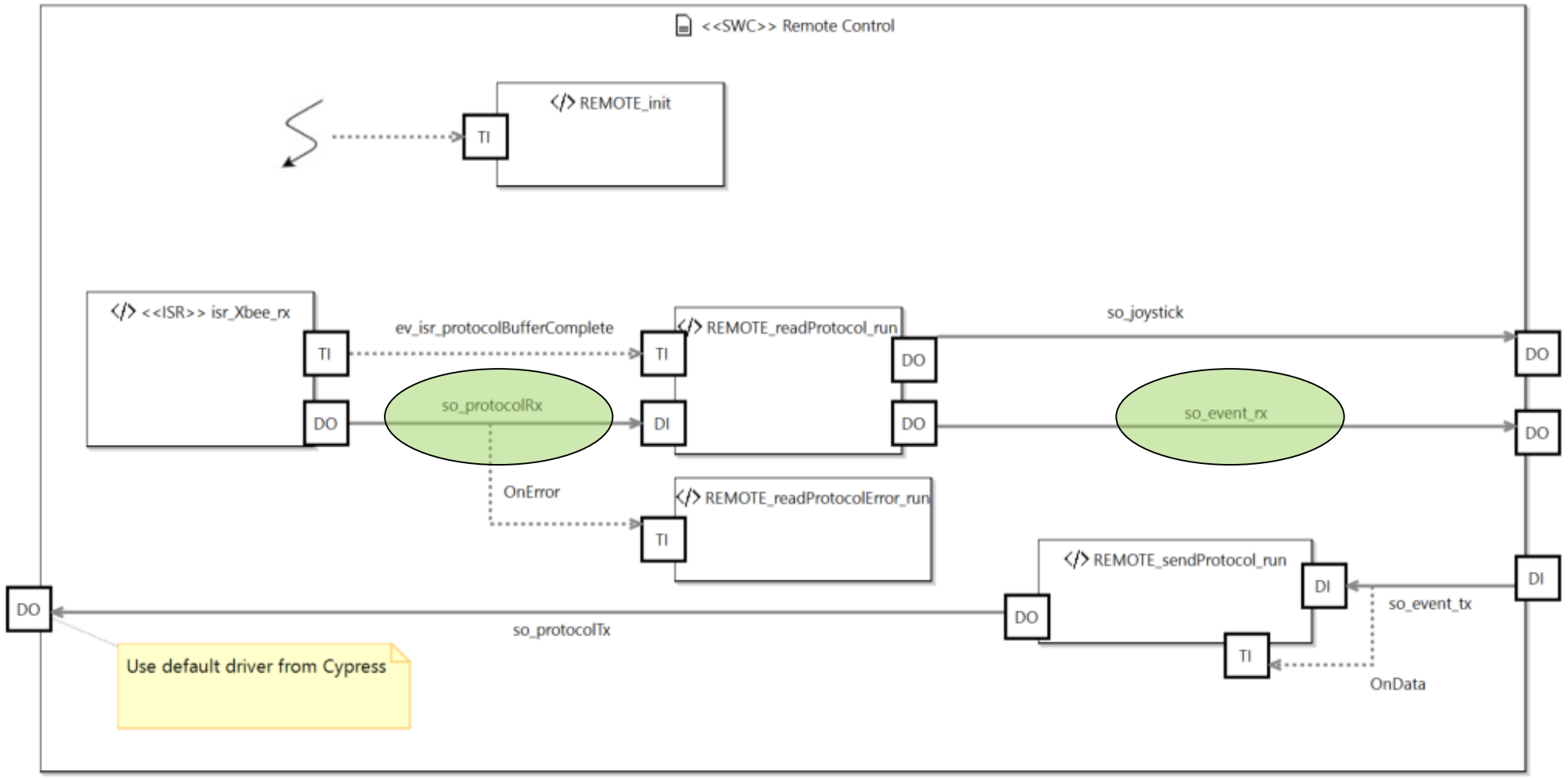
Power on reset per power domain possible

State Machine Implementation

- Generic state machine implementation using 2 layer lookup table pattern
- Configuration of state and event type through configuration file
- Init function added to allow power on entry actions

```
/** ---------------------------------- Transitions ---------------------------------------- **/

/**  ===== IS_IDLE ====   */
static const STATE_stateInnerTransitionTable_t CONTROL_ISIDLE_Transitions = {
/*    Event                ToState              Guard                Action                      */
    { EV_Connect,          IS_CONNECTED,        0,                   CONTROL__action_confirmConnect},
};

/**  ===== IS_CONNECTED ====   */
static const STATE_stateInnerTransitionTable_t CONTROL_ISCONNECTED_Transitions = {
/*    Event                ToState              Guard                Action                      */
    { EV_Reset,            IS_IDLE,             0,                   0},
    { EV_Disconnect,       IS_IDLE,             0,                   CONTROL__action_confirmDisconnect},
    { EV_RequestFeature,   IS_CONNECTED,        0,                   CONTROL__action_provideFeatureList},
    { EV_ActivateFeature,  IS_DRIVING,          0,                   CONTROL__action_activateFeature},
};

/**  ===== IS_DRIVING ====   */
static const STATE_stateInnerTransitionTable_t CONTROL_ISDRIVING_Transitions = {
/*    Event                ToState              Guard                Action                      */
    { EV_Joystick,         IS_DRIVING,          0,                   CONTROL__Drive},
    { EV_Reset,            IS_IDLE,             0,                   0},
    { EV_Disconnect,       IS_IDLE,             0,                   CONTROL__action_confirmDisconnect),
};


static const STATE_stateOuterTransitionTable_t CONTROL_Connection_Transitions = {
/*    fromState         Pointer to table                    Size of table [Elements]    */
    { IS_IDLE,          &CONTROL_ISIDLE_Transitions,        sizeof(CONTROL_ISIDLE_Transitions)/sizeof(STATE_stateInnerTransition_t)    },
    { IS_CONNECTED,     &CONTROL_ISCONNECTED_Transitions,   sizeof(CONTROL_ISCONNECTED_Transitions)/sizeof(STATE_stateInnerTransition_t)     }
    { IS_DRIVING,       &CONTROL_ISDRIVING_Transitions,     sizeof(CONTROL_ISDRIVING_Transitions)/sizeof(STATE_stateInnerTransition_t) },
};
static const uint16_t CONTROL_Connection_Transitions_size = sizeof(CONTROL_Connection_Transitions)/sizeof(STATE_stateOuterTransition_t);

/** ---------------------------------- Action ---------------------------------------- **/

static const STATE_stateActionTable_t CONTROL_Connection_StateActions = {
/*  fromState           Entry                       Exit    */
  { IS_IDLE,            CONTROL__action_stopCar,    0   }
};
static const uint16_t CONTROL_Connection_StateActions_size = sizeof(CONTROL_Connection_StateActions)/sizeof(STATE_stateAction_t);
```

Communication Sequence



ZigBee Protocol versus Event

```
#define PROT_SIZEPAYLOAD 40             /**< \brief Payload size (max) */
#define PROT_SIZERXBUF   100            /**< \brief Size of the UART reception buffer */
#define PROT_EOP1        0x0D           /**< \breif EOP character 1 */
#define PROT_EOP2        0x0E           /**< \breif EOP character 2 */

typedef struct
{
    PROT_id_t       m_id;                       /**< \brief Protocol ID, first byte in protocol */
    PROT_sender_t   m_sender;                   /**<\ brief identifier of the sender - mainly for debugging */
    PROT_fid_t      m_fid;                      /**<\ brief feature identifier */
    uint8_t         m_length;                   /**<\ length of the valid payload */
    uint8_t         m_payload[PROT_SIZEPAYLOAD];    /**<\ payload */
} SIGNAL_PROTOCOL_data_t;
```

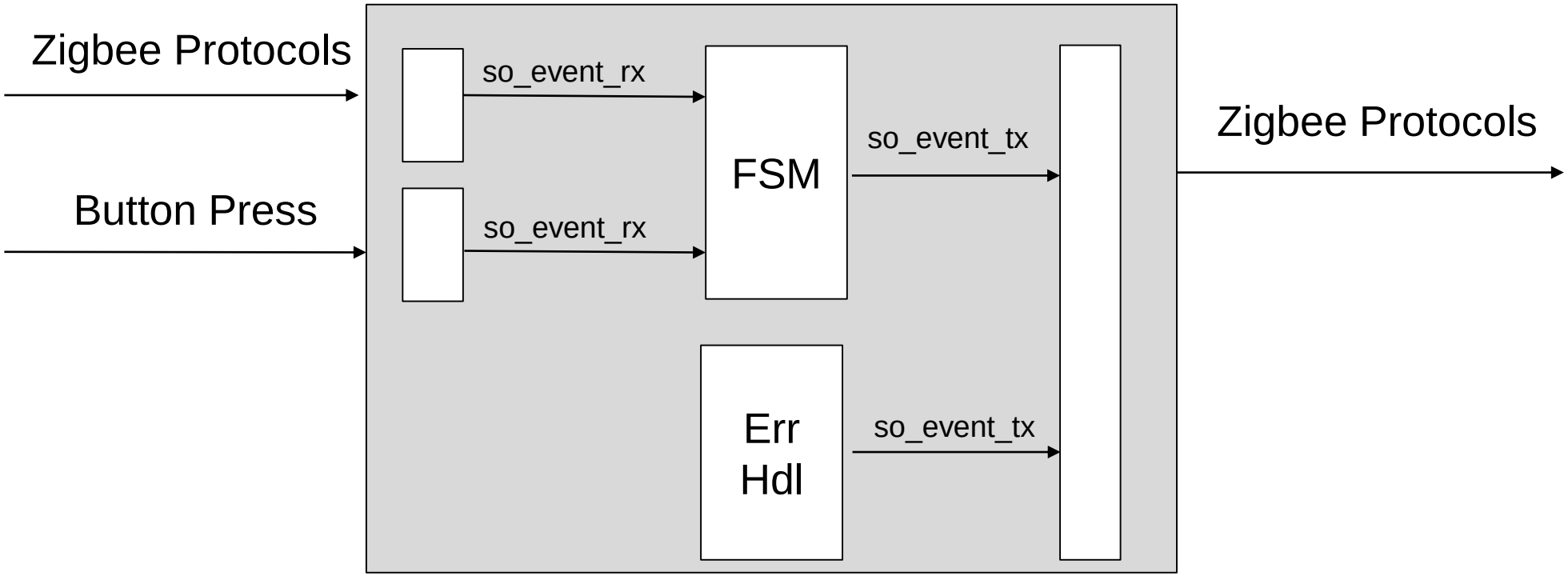
```
#define EVENT_PAYLOADSIZE PROT_SIZEPAYLOAD

typedef struct
{
    EVENT_t m_ev;
    uint8_t m_lengthPayload;
    uint8_t m_payload[EVENT_PAYLOADSIZE];
} SIGNAL_EVENT_data_t;


#define SIGNAL_EVENT_INIT_DATA ((SIGNAL_EVENT_data_t){0})
```

Remote Code

```
void REMOTE_sendProtocol_run()
{
    //Read in internal message
    SIGNAL_EVENT_data_t event = SIGNAL_EVENT_INIT_DATA;
    event = RTE_SIGNAL_EVENT_get(&SO_EVENT_TX_signal);

    //Translate message into protocol
    SIGNAL_PROTOCOL_data_t prot = SIGNAL_PROTOCOL_INIT_DATA;

    if (EV_Feature_List == event.m_ev)
    {
        //Activate feature
        prot.m_id = PROT_ID_Connection;
        prot.m_sender = PROT_sender_car;
        prot.m_fid = PROT_FID_Feature;
        prot.m_length = event.m_lengthPayload;

        //Copy payload
        for (uint8_t i = 0; i < prot.m_length; i++)
        {
            prot.m_payload[i] = event.m_payload[i];
        }

        LOG_I("REMOTE_sendProtocol_run","Sending Feature : %d", prot.m_payload[0]);
    }
    else
    {
        //All other simple events without payload
        prot.m_id = PROT_ID_Connection;
        prot.m_sender = PROT_sender_car;
        switch (event.m_ev)
        {
            case EV_Connect_ACK : prot.m_fid = PROT_FID_Connect; break;
            case EV_Disconnect_ACK : prot.m_fid = PROT_FID_Disconnect; break;
            default: LOG_E("REMOTE_sendProtocol_run","wrong event: %d",event.m_ev); break;
        }
    }

    RTE_SIGNAL_PROTOCOL_set(&SO_PROTOCOLTX_signal, prot);
    RTE_SIGNAL_PROTOCOL_pushPort(&SO_PROTOCOLTX_signal);

}
```

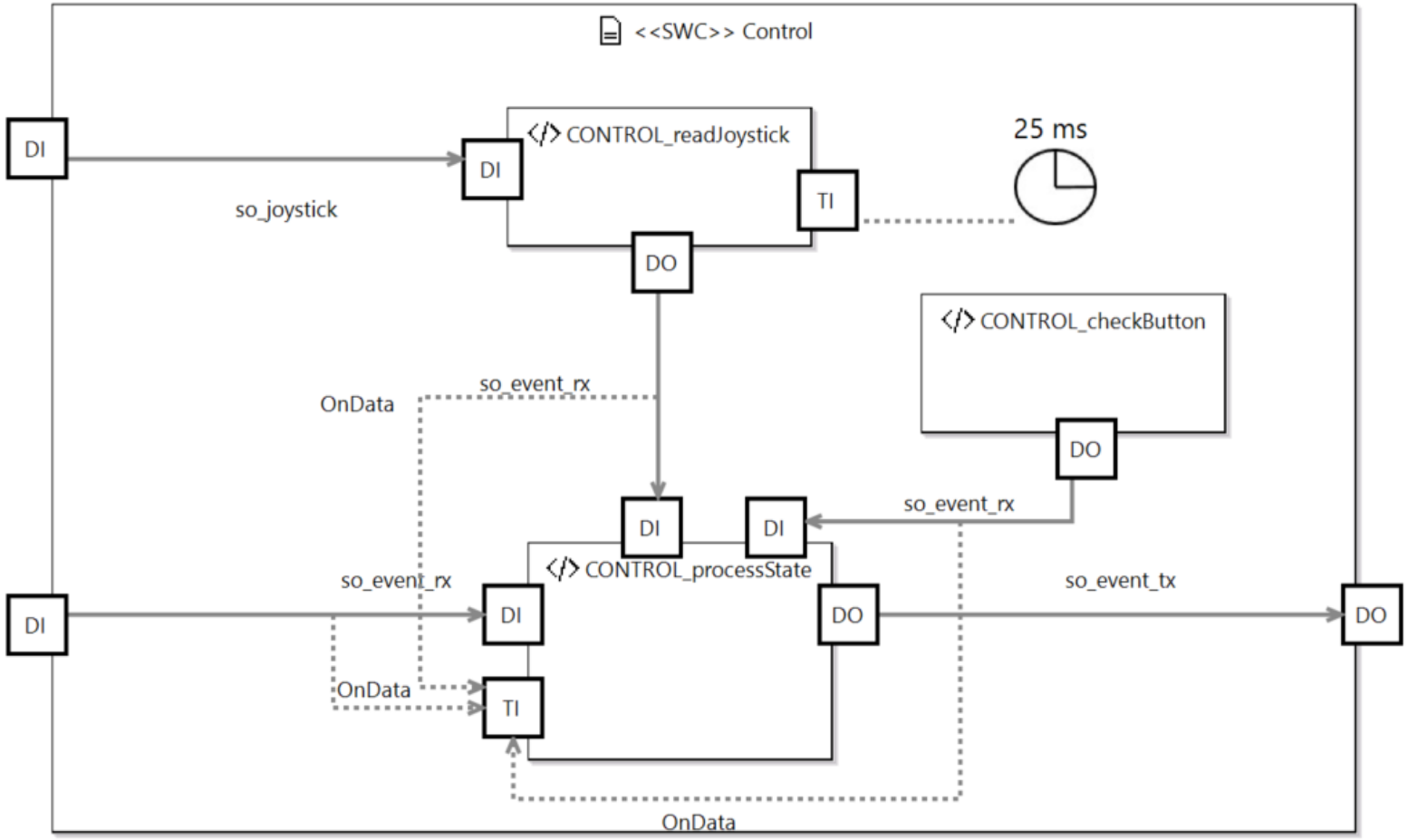
Do we really need 2 different signals? Can't the application work directly with protocols?

Signalflow from UART to Statemachine on the Remote side



Event signals can be used as flexible, internal communication / message objects with integrated trigger function. Don’t mix them up with ERIKA events – different story.

Resulting Signal flow on Remote side



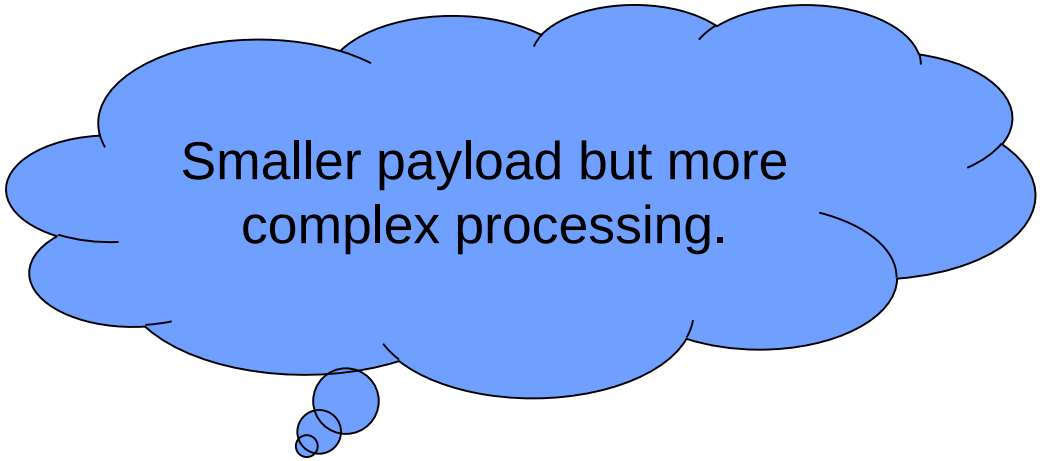
Actions – same structure for all of them

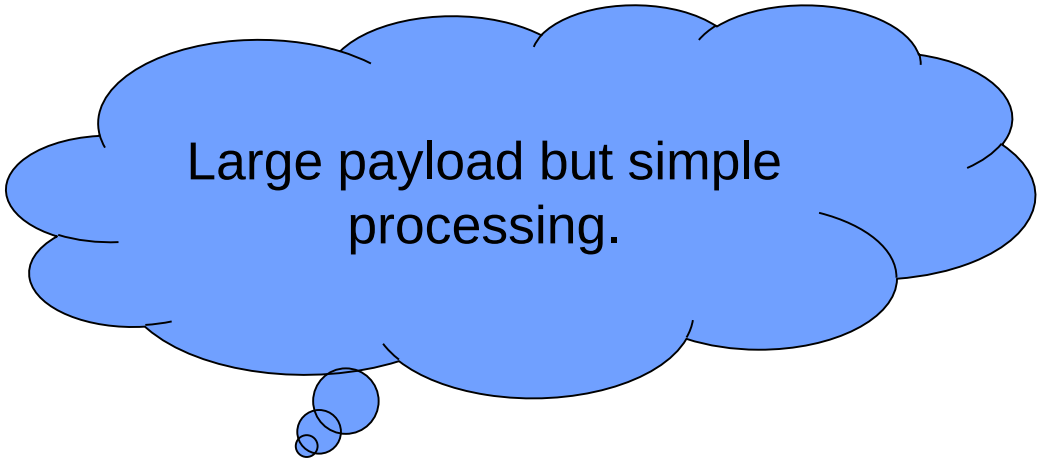
```
/**
 * Will send an acknowledge protocol
 */
static void CONTROL__action_confirmConnect()
{
    SIGNAL_EVENT_data_t ev = SIGNAL_EVENT_INIT_DATA;

    ev.m_ev = EV_Connect_ACK;

    RTE_SIGNAL_EVENT_set(&SO_EVENT_TX_signal, ev);
}

/**
 * Will send an acknowledge protocol
 */
static void CONTROL__action_confirmDisconnect()
{
    //This protocol is ignored on remote side, may be added later for checking
    SIGNAL_EVENT_data_t ev = SIGNAL_EVENT_INIT_DATA;

    ev.m_ev = EV_Disconnect_ACK;

    RTE_SIGNAL_EVENT_set(&SO_EVENT_TX_signal, ev);
}
```

Transmission of the feature list

We have 2 options

- Either transmit all features in a single protocol
- Or one feature per protocol

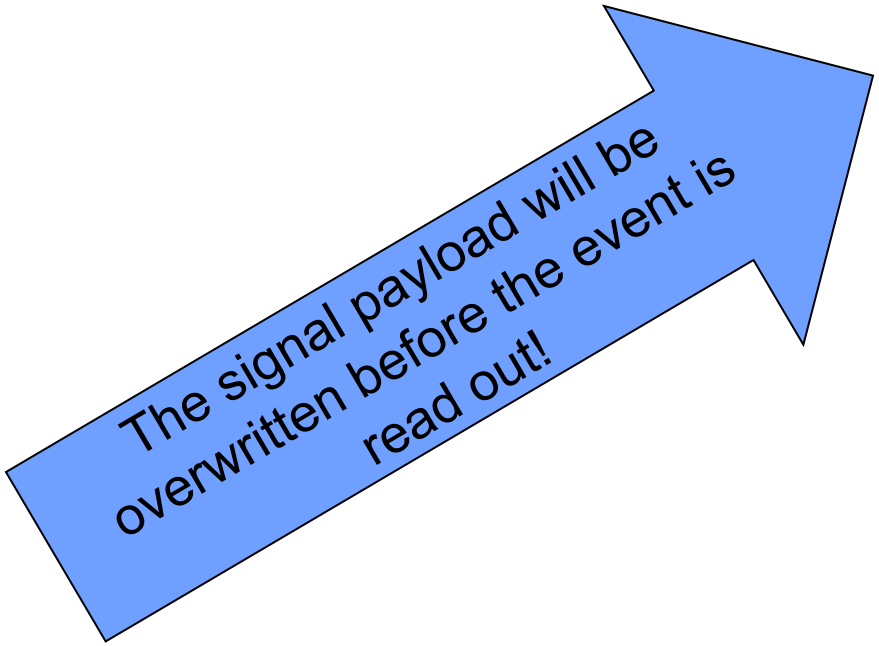




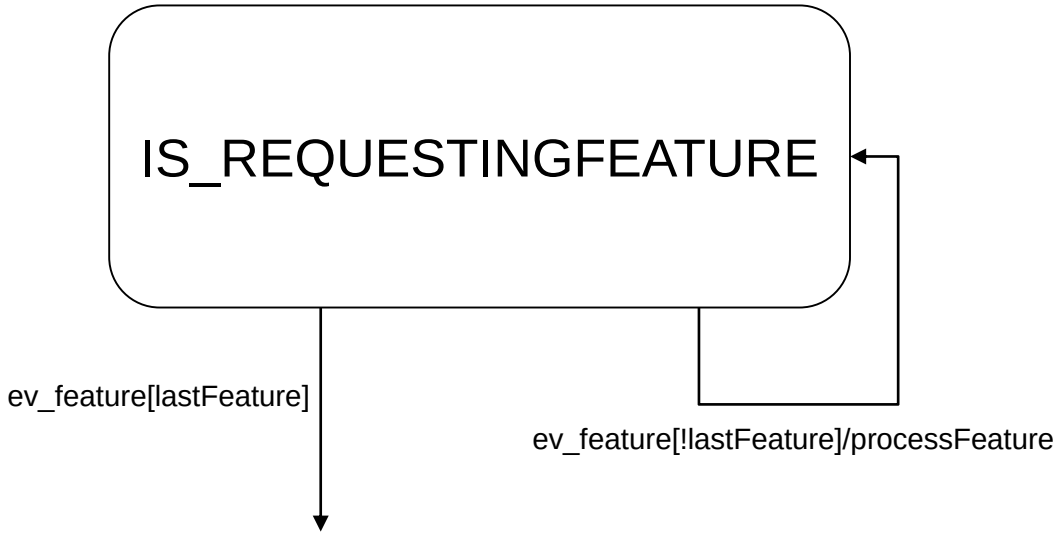
RTE limitation – signals are not queued

Sending Side

For each feature
    event.payload=featureName
    RTE_set(&so_event, event)



Reception Side



Feature List Packing – Single Protocol Transfer

```
static void CONTROL__action_provideFeatureList()
{
    SIGNAL_EVENT_data_t ev = SIGNAL_EVENT_INIT_DATA;
    ev.m_ev = EV_Feature_List;

    uint8_t pos = 0;

    //Stringnify the existing features
    for (uint8_t i = 0; i < CONTROL_FeatureTable_size; i++)
    {
        LOG_I("CONTROL__action_provideFeatureList","Processing Feature %d of %d", i, CONTROL_FeatureTable_size);

        uint8_t len = strlen(CONTROL_FeatureTable[i].m_description);

        //Should not happen, but who knows
        if (len >= CONTROL_FEATURESTRINGLENGTH) len = CONTROL_FEATURESTRINGLENGTH;

        //Copy string content
        for (uint8_t l = 0; l < len; l++)
        {
            ev.m_payload[pos++] = CONTROL_FeatureTable[i].m_description[l];
            if (pos >= EVENT_PAYLOADSIZE)
            {
                LOG_E("CONTROL__action_provideFeatureList","payload too large");
                return;
            }

        }
        ev.m_payload[pos++] = '\0'; //String termination
        if (pos >= EVENT_PAYLOADSIZE)
        {
            LOG_E("CONTROL__action_provideFeatureList","payload too large");
            return;
        }
    }

    //Set resulting payload size
    ev.m_lengthPayload = pos;

    //Send data
    RTE_SIGNAL_EVENT_set(&SO_EVENT_TX_signal, ev);
}
```

Feature List Unpacking

```
static void CONTROL__action_processFeatureList()
{
    //Read in event to check payload
    SIGNAL_EVENT_data_t ev = RTE_SIGNAL_EVENT_get(&SO_EVENT_RX_signal);

    //The display signal needs to be updated
    SIGNAL_DISPLAY_data_t display = RTE_SIGNAL_DISPLAY_get(&SO_DISPLAY_signal);

    //clear the features
    display.m_numberFeatures = 0;
    display.m_selectedFeature = 0;

    for (uint8_t i = 0; i < DISPLAY_NUMBERFEATURE;i++)
    {
        strcpy(display.m_feature[i],"");
    }

    //Just to be on the safe side
    if (EV_Feature_List == ev.m_ev)
    {
        uint8_t len = ev.m_lengthPayload;
        uint8_t datapos = 0;

        char data[PROT_SIZEPAYLOAD];

        for (uint8_t l = 0; l < len; l++)
        {

            //LOG_I("- "," %d %d %c",l, ev.m_payload[l], ev.m_payload[l]);
            data[datapos++] = ev.m_payload[l];

            //Next element reached
            if (0 == ev.m_payload[l])
            {
                datapos = 0;

                //Copy feature to next extry of
                if (display.m_numberFeatures < DISPLAY_NUMBERFEATURE)
                {
                    strcpy(display.m_feature[display.m_numberFeatures++],data);
                    LOG_I("CONTROL__action_processFeatureList","Feature: %s",data);
                }
            }
        }
    }

    //Write signal to RTE
    RTE_SIGNAL_DISPLAY_set(&SO_DISPLAY_signal, display);
}
```

Integrating the Display

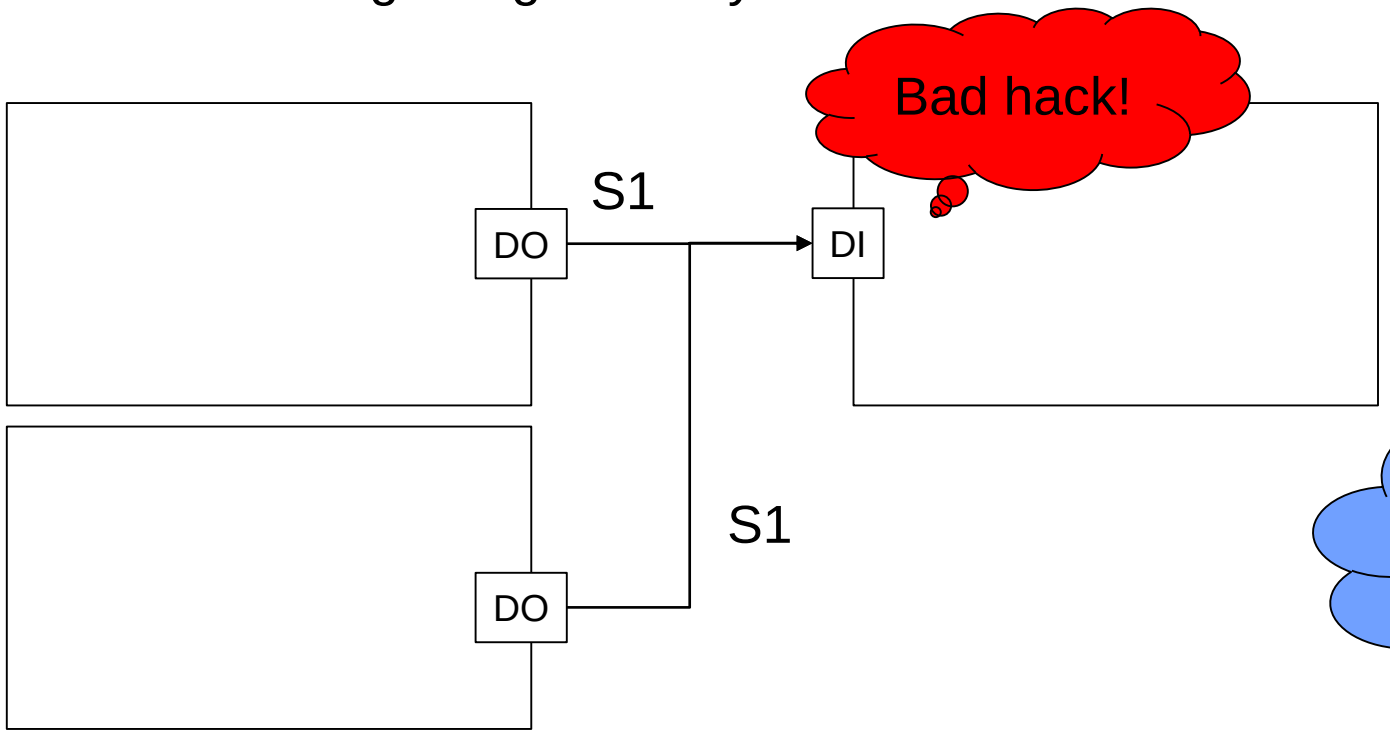
- The display signal is used to store the view model of the remote control
- Pushport is used to send the data to the display

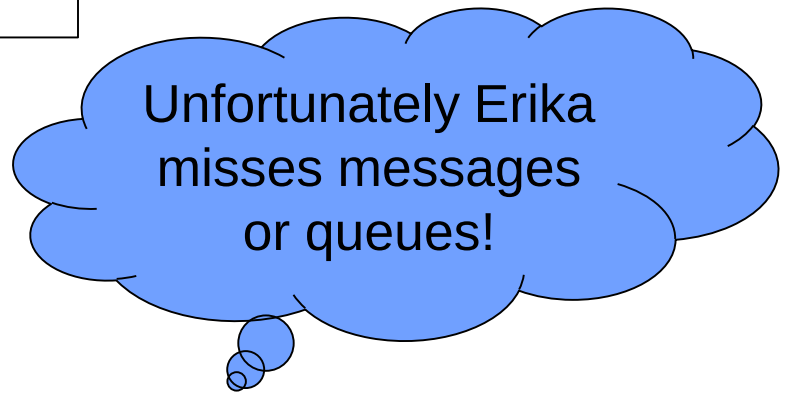
```
inline RC_t SIGNAL_DISPLAY_driverOut(const SIGNAL_DISPLAY_data_t  *const data)
{

    TFT_clearScreen();

    TFT_setCursor(0,0);
    TFT_setTextColor(WHITE);

    LED_Set(LED_ALL, LED_OFF);

    //Scale application data to drive format
    switch (data->m_state)
    {
        case IS_IDLE :              TFT_print("      IS_IDLE"); LED_Set(LED_RED, LED_ON); break;
        case IS_REQUESTINGCONNECT : TFT_print(" IS_REQUESTINGCONNECT"); LED_Set(LED_YELLOW, LED_ON);break;
        case IS_CONNECTED :         TFT_print("   IS_CONNECTED"); LED_Set(LED_YELLOW, LED_ON);break;
        case IS_REQUESTINGFEATURE : TFT_print(" IS_REQUESTINGFEATURE"); LED_Set(LED_YELLOW, LED_ON);break;
        case IS_SELECTINGFEATURE :  TFT_print(" IS_SELECTINGFEATURE"); LED_Set(LED_YELLOW, LED_ON);break;
        case IS_DRIVING :           TFT_print("    IS_DRIVING"); LED_Set(LED_GREEN, LED_ON);break;
    }

    TFT_writeLine(0,10,200,10,WHITE);

    for (uint8_t i = 0; i < data->m_numberFeatures; i++)
    {
        TFT_setCursor(20,(i+2)*10);
        TFT_print((char*)data->m_feature[i]);
    }

    //Show the cursor
    if (data->m_numberFeatures > 0)
    {
        TFT_fillTriangle(2, (data->m_selectedFeature+3)*10-2, 16, (data->m_selectedFeature+2)*10 + 5 - 2, 2, (data->m_selectedFeature+2)*10
    }
    else
    {
        TFT_setCursor(20,20);
        TFT_setTextColor(RED);
        TFT_print("No Features yet");
    }

    //Write scaled data to driver

    return RC_SUCCESS;
}
```

Limitations of the RTE

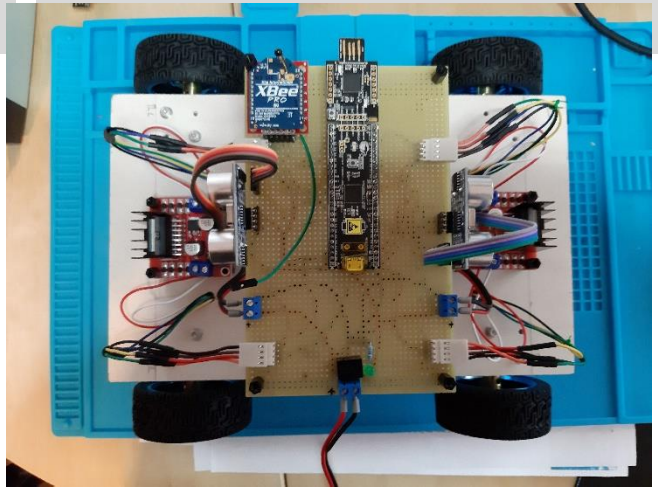
- In the old release of the RTE, we had only one cyclic and one event task. This caused problems with long running functions. ➔ Flexible mapping of runnables to tasks. Leading to a more complex event dispatcher.

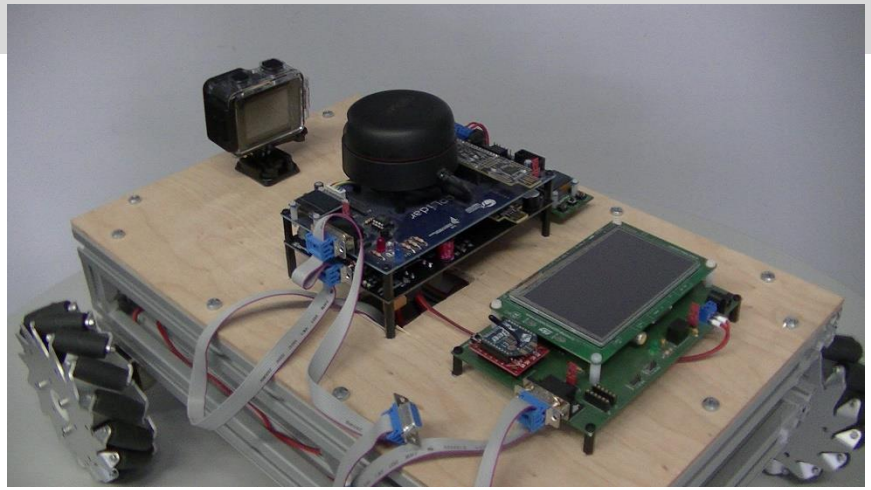
- Signals are not queued and therefore not threadsafe.
- Also a too fast sending of signals may cause races

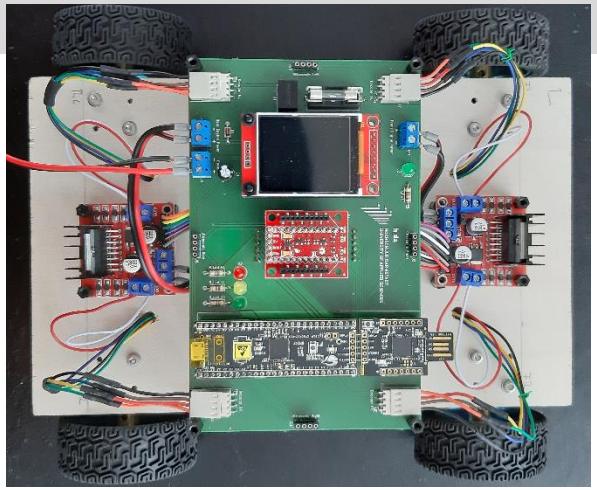




Migration Story







CAR 6 v1 Hardware

20 mandays

10 mandays (HAL)

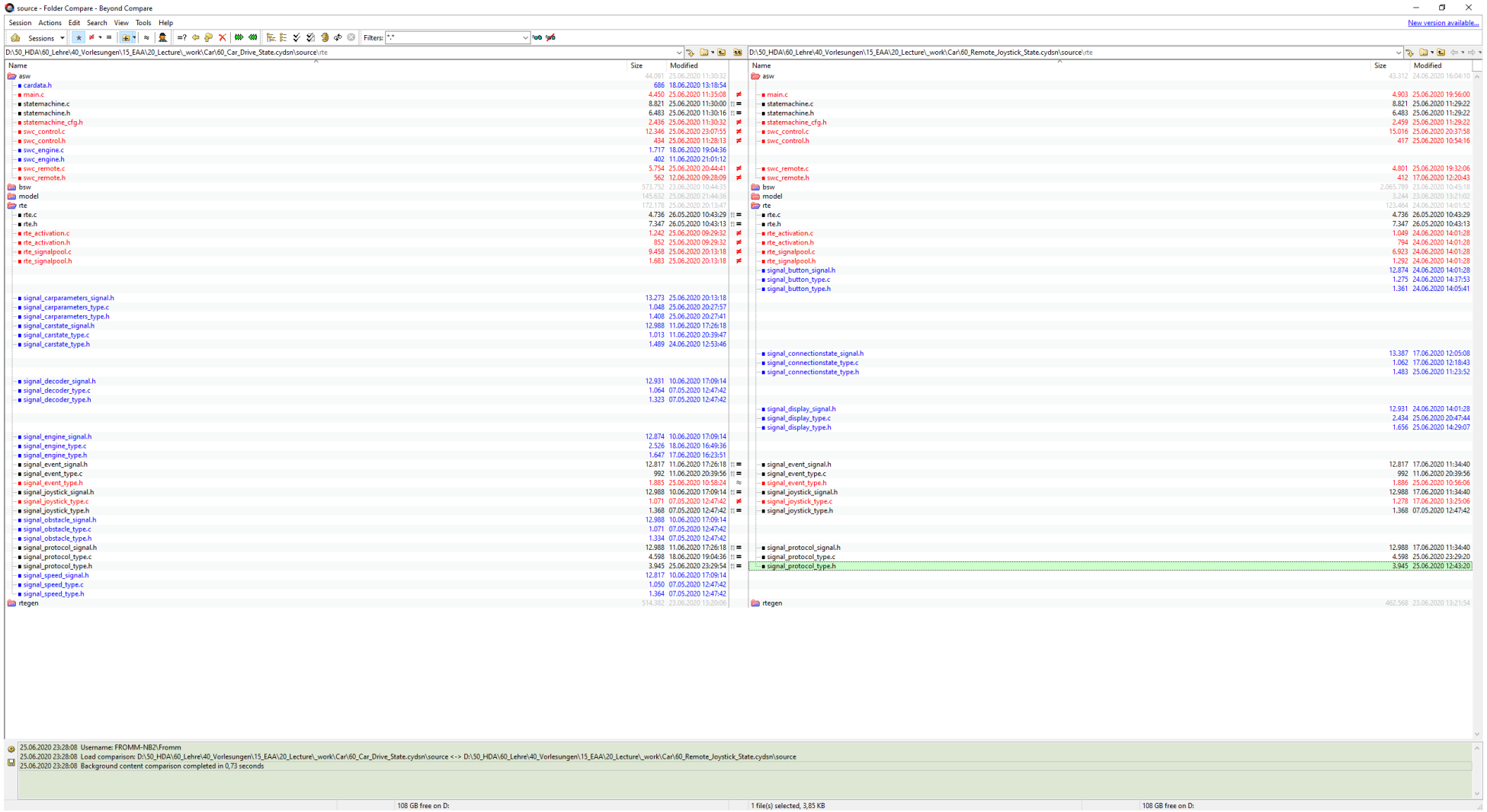
2 mandays

CAR 6 v1 RTE

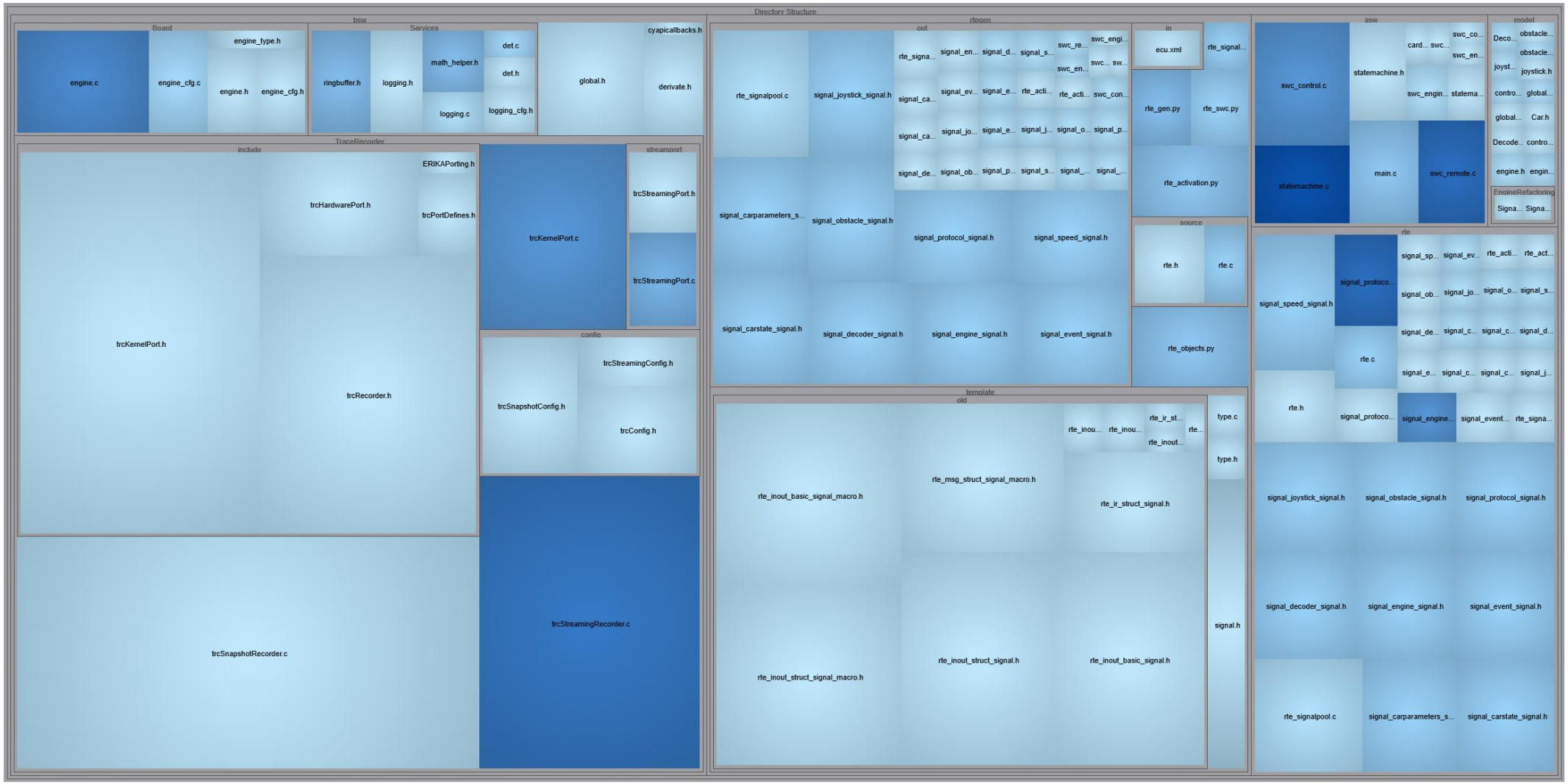
CAR 6 v1 Application → CAR 5 Application → CAR 6 v2 Application

CAR 6 v1 Remote → CAR 5 Remote

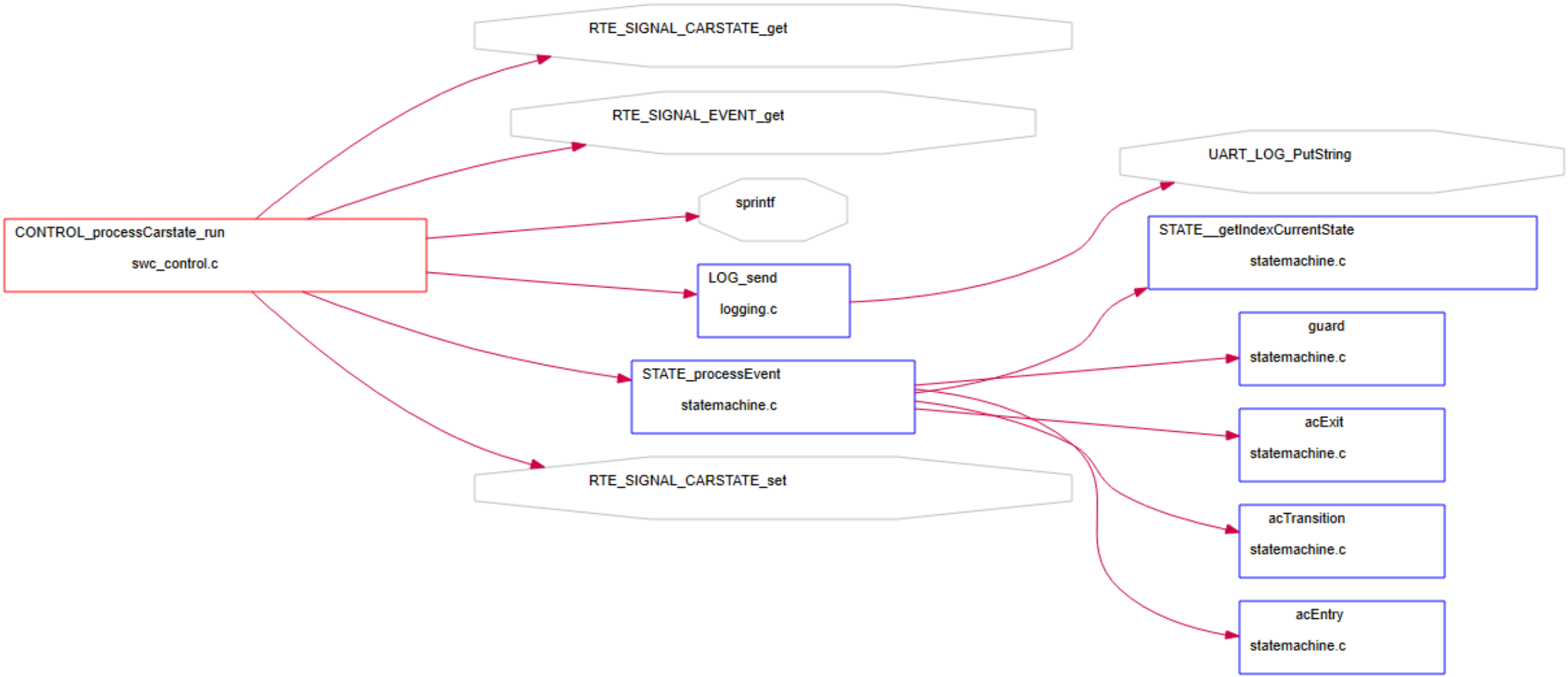
Quite some parts could be reused



Metrics


Directory Structure
bsw
Board
engine.c
engine_cfg.c
engine_type.h
engine.h
engine_cfg.h
Services
ringbuffer.h
logging.h
math_helper.h
logging.c
det.c
det.h
logging_cfg.h
cyapicallbacks.h
global.h
derivate.h
TraceRecorder
include
ERIKAPorting.h
trcHardwarePort.h
trcPortDefines.h
trcKernelPort.h
trcRecorder.h
trcKernelPort.c
streamport
trcStreamingPort.h
trcStreamingPort.c
config
trcStreamingConfig.h
trcSnapshotConfig.h
trcConfig.h
trcSnapshotRecorder.c
trcStreamingRecorder.c
rtegen
out
rte_signalpool.c
signal_joystick_signal.h
signal_carparameters_s...
signal_obstacle_signal.h
signal_protocol_signal.h
signal_speed_signal.h
signal_carstate_signal.h
signal_decoder_signal.h
signal_engine_signal.h
signal_event_signal.h
in
ecu.xml
rte_gen.py
rte_swc.py
rte_activation.py
source
rte.h
rte.c
rte_objects.py
template
old
rte_inout_basic_signal_macro.h
rte_msg_struct_signal_macro.h
rte_ir_struct_signal.h
rte_inout_struct_signal_macro.h
rte_inout_struct_signal.h
rte_inout_basic_signal.h
type.c
type.h
signal.h
asw
swc_control.c
statemachine.h
statemachine.c
main.c
swc_remote.c
model
EngineRefactoring
rte
signal_speed_signal.h
signal_protocol_signal.h
rte.c
rte.h
signal_engine_signal.h
signal_joystick_signal.h
signal_obstacle_signal.h
signal_protocol_signal.h
signal_decoder_signal.h
signal_engine_signal.h
signal_event_signal.h
rte_signalpool.c
signal_carparameters_s...
signal_carstate_signal.h

Loose coupling



Wrap-Up

- Although the project has a certain complexity, the complete application code for both the car and the remote control was written in less than 2 days.

- Consequent usage of design and coding pattern.
- Not too many hacks.

- Dependent files between both projects (e.g. protocol definition, state events,…) share exactly the same code base.

- Debugging was mainly done using UART-LOG.